

Отчёта по лабораторной работе 7

Имитационное моделирование

Нджову Нелиа

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Цель этой лабораторной работы - работа дискретно-событийного моделирования

2 Задание

1. Модель М/М/с
2. Модель Росса

3 Теоретическое введение

Модель М/М/с

Модель М/М/с(по классификации Кендалла) - это система массового обслуживания со следующими свойствами:

- М(Markovian) - входящий поток пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ .
- М - время обслуживание каждой заявки распределено экспоненциально с параметром μ .
- с - количество идентичных обслуживающих приборов(каналов), работающих параллельно.

Основные параметры

- Интенсивность входящего потока(λ) - Среднее число заявок в единицу времени. Интервалы прибытия $\sim \text{Exp}(1/\lambda)$
- Интенсивность обслуживания(одного канала)(μ) - Среднее число заявок, обслуживаемых одним прибором в единицу времени. Длительность обслуживания $\sim \text{Exp}(1/\mu)$
- Число каналов(с) - Количество параллельных серверов
- Загрузка системы на одним канал(traffic intensity)($\rho = \lambda/(c \cdot \mu)$) - Условие стационарности: $\rho < 1$

Отличия от модели M/M/1

Характеристика	M/M/1	M/M/c
Число каналов	1	$c (c > 1)$
Поведение заявки	Ждёт единственный канал	Ждёт любой свободный канал из c
Условие стационарности	$\lambda < \mu$	$\lambda < c\mu$
Формулы для W_q	$\rho/\mu(1-\rho)$	Более сложные
Вероятность ожидания	Равна ρ	Определяется формулой Эрланта

Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

Формальная постановка задачи

— В системе находятся N идентичных машин, которые постоянно работают и могут выходить из строя.

— S машин находятся в резерве и готовы немедленно заменить любую отказавшую.

— Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.

— Когда работающая машина ломается, происходит следующее:

— Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломанной.

— Сломанная машина отправляется в ремонт.

— Если резерва нет, система падает (crash). Моделирование заканчивается.

— После ремонта машина пополняет пул резервных (становится исправной и ждёт).

— Требуется оценить среднее время до падения системы $E[T]$ при заданных распределениях наработки до отказа и времени ремонта.

Марковская модель

Данная система может быть описана как марковский процесс с непрерывным временем и конечным числом состояний. Состояние можно определить как число работоспособных машин (работающих плюс резервных) или число машин в ремонте.

Пусть k — число исправных машин, $0 \leq k \leq N + S$. Классический подход из учебника Росса: состояние — число исправных машин, а число работающих всегда равно N , если есть хотя бы N исправных.

Состояния:

— i — число исправных машин, $i = 0, 1, \dots, N + S$.

— Число работающих машин всегда $\min(N, i)$.

— Число резервных машин — $\max(0, i - N)$.

— Число машин в ремонте — $N + S - i$.

Интенсивности переходов:

— При отказе машины (если есть работающая) число исправных машин уменьшается на 1. Интенсивность: $\min(N, i)\lambda$ (если $i \geq 1$).

— При завершении ремонта число исправных увеличивается на 1. Интенсивность: μ , если есть хотя бы одна машина в ремонте (т.е. $i < N + S$). Ремонтник один, поэтому интенсивность постоянна, когда очередь на ремонт не пуста.

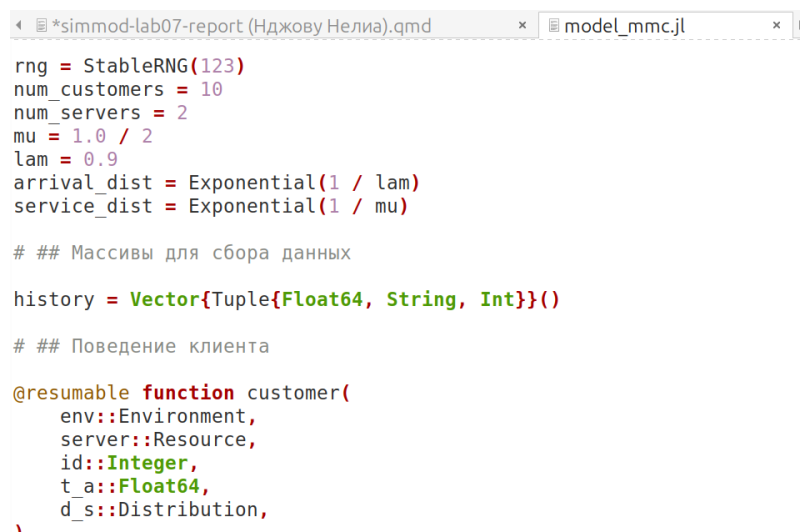
Система падает, когда число исправных машин $i = N - 1$ (если ещё есть работающая) или $i = N$. Падение происходит в момент отказа, когда нет резерва. Это соответствует переходу из состояния N (0 резерва, все N машин работают) в неработоспособное состояние. В момент отказа одной из N машин при $i = N$ запасных нет, следовательно, система падает. Значит, падение наступает при первом достижении состояния $i = N - 1$ (при переходе $i = N \rightarrow N - 1$).

Аналитически среднее время до отказа можно найти, решив систему линейных уравнений для средних времен достижения поглощающего состояния.

4 Выполнение лабораторной работы

4.1 1. Модель М/М/с

Я создала файл `scripts/model_mmc.jl` и скопировала заданный скрипт в него. Скрипт создает 10 клиентов в очереди из 2 серверов и выводит время прибытия, начала обслуживания и отправления в консоль. Я запускала код и все работало хорошо(рис.1).



```
*simmod-lab07-report (Нджову Нелиа).qmd  model_mmc.jl

rng = StableRNG(123)
num_customers = 10
num_servers = 2
mu = 1.0 / 2
lam = 0.9
arrival_dist = Exponential(1 / lam)
service_dist = Exponential(1 / mu)

# ## Массивы для сбора данных

history = Vector{Tuple{Float64, String, Int}}{0}

# ## Поведение клиента

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)
```

Рисунок 4.1: Заданный скрипт

После этого я перевожу код в структуру DrWatson(рис.2)

```
# ## Загрузка необходимых пакетов
```

```
using DrWatson  
@quickactivate "project"
```

Рисунок 4.2: Структура DrWatson

Затем я добавляю необходимую визуализацию(рис.3)

```
simmod-lab07-report (Нджову Нелиа).qmd × model_mmc.jl × model_ross.jl ×  
p1 = plot(df_results.time, df_results.n_in_system,  
    label = "Клиентов в системе",  
    xlabel = "Время",  
    ylabel = "Количество клиентов",  
    title = "М/М/с: Загрузка системы во времени",  
    linewidth = 2,  
    legend = :topright)  
display(p1)  
savefig(plotsdir("mmc_system_load.png"))  
  
# ## График 2: Длина очереди во времени  
p2 = plot(df_results.time, df_results.queue_length,  
    label = "Длина очереди",  
    xlabel = "Время",  
    ylabel = "Клиентов ожидает",  
    title = "М/М/с: Динамика очереди",  
    linewidth = 2,  
    legend = :topright)  
display(p2)  
savefig(plotsdir("mmc_queue_length.png"))  
  
# ## График 3: Использование серверов во времени  
ρ = lam / (num_servers * mu)  
p3 = plot(df_results.time, df_results.utilization,  
    label = "Использование",  
    xlabel = "Время",  
    ylabel = "Использование (0-1)",  
    title = "М/М/с: Загрузка серверов",  
    linewidth = 2,  
    legend = :topright,  
    ylims = (0, 1))  
hline!([ρ], label = "Теоретический ρ", linestyle = :dash)
```

Рисунок 4.3: Визуализация

На первом графике показано количество клиентов в очереди, из графика видно, что наибольшее количество клиентов в очереди находилось в интервале 5-10 единиц времени(рис.4)

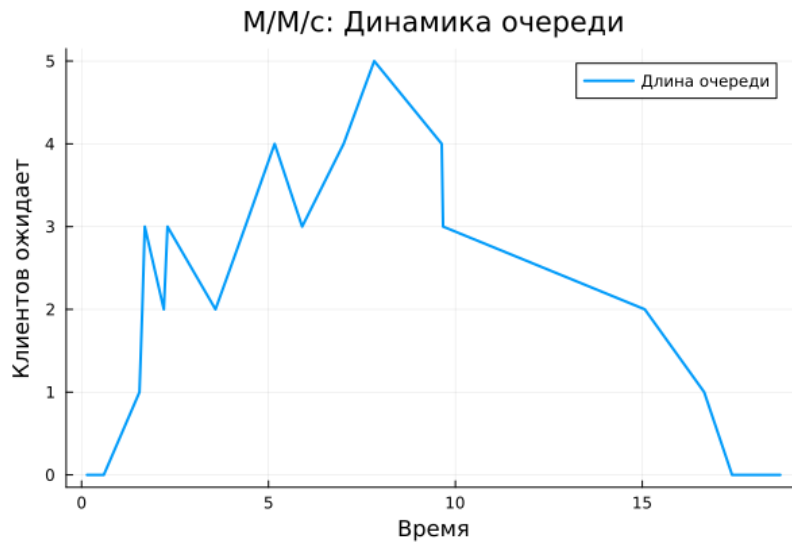


Рисунок 4.4: Динамика очереди

Второй график, который я добавил, был графиком количества клиентов в системе, из графика, аналогичного первому, мы можем видеть, что наибольшее количество клиентов находится в пределах 5-10 единиц времени(рис.5)

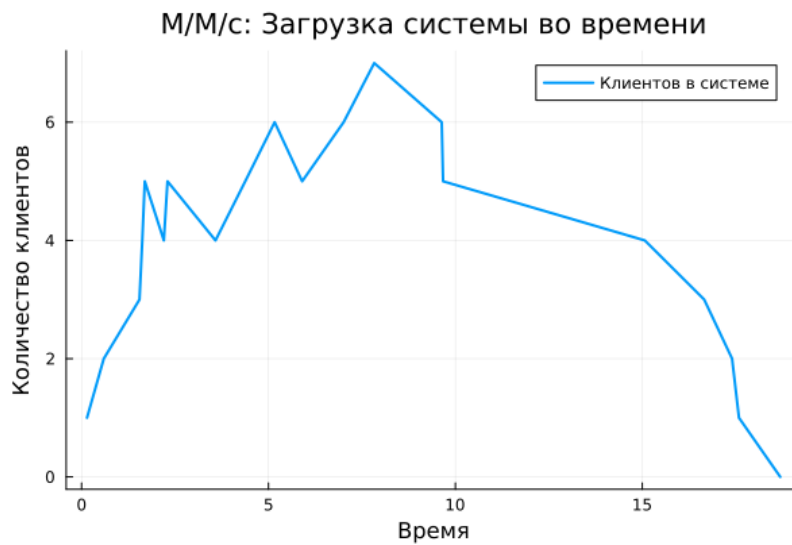


Рисунок 4.5: Загрузка системы во времени

Третий график предназначен для проверки того, насколько загружен сервер по сравнению с теоретическим порогом (ρ), из графика мы видим, что сервер загружен больше, чем ожидалось(рис.6)

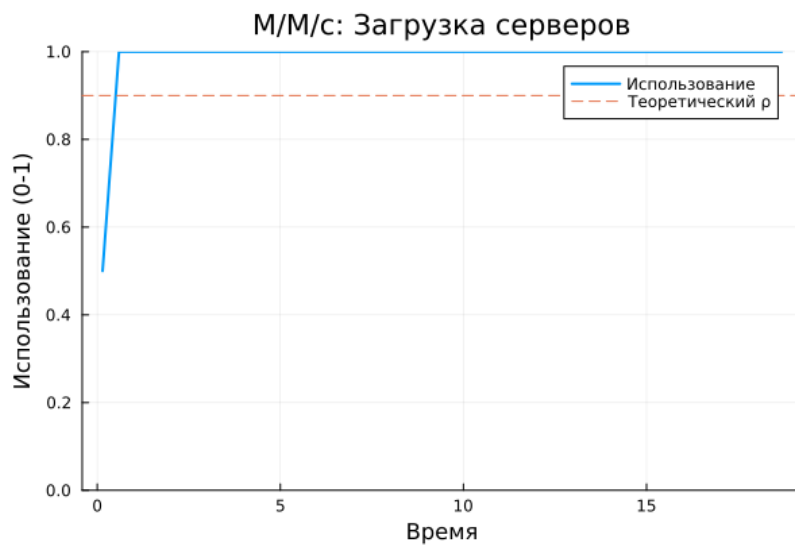


Рисунок 4.6: Загрузка серверов

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/model_mmc.qmd`

5 Модель массового обслуживания

M/M/c

Данная симуляция моделирует многоканальную систему массового обслуживания с пуассоновским входящим потоком (интенсивность λ) и экспоненциальным временем обслуживания (интенсивность μ). Клиенты поступают, ожидают в очереди FIFO, если все серверы заняты, затем обслуживаются и покидают систему.

5.1 Загрузка необходимых пакетов

```
using DrWatson
@quickactivate "project"

using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions
using DataFrames
using Plots
using CSV
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find find a project file by recursively checking given `dir`  
| (given dir: /home/nelianjovu)  
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
Plots.GRBackend()
```

5.2 Параметры симуляции

```
rng = StableRNG(123)  
num_customers = 10  
num_servers = 2  
mu = 1.0 / 2  
lam = 0.9  
arrival_dist = Exponential(1 / lam)  
service_dist = Exponential(1 / mu)
```

```
Exponential{Float64}( $\lambda$ =2.0)
```

5.3 Массивы для сбора данных

```
history = Vector{Tuple{Float64, String, Int}}()
```

```
Tuple{Float64, String, Int64}[]
```

5.4 Поведение клиента


```

@resumable function customer(
    env::Environment,
    server::Resource,
    id::Integer,
    t_a::Float64,
    d_s::Distribution,
)

    @yield timeout(env, t_a)
    println("Customer $id arrived: ", now(env))
    push!(history, (now(env), "arrived", id))

    @yield request(server)
    println("Customer $id entered service: ", now(env))
    push!(history, (now(env), "started", id))

    @yield timeout(env, rand(rng, d_s))
    @yield unlock(server)
    println("Customer $id exited service: ", now(env))
    push!(history, (now(env), "exited", id))
end

```

customer (generic function with 1 method)

5.5 Запуск симуляции

```

function setup_and_run()
    sim = Simulation()
    server = Resource(sim, num_servers)

```

```

    arrival_time = 0.0

    for i = 1:num_customers
        arrival_time += rand(rng, arrival_dist)
        @process customer(sim, server, i, arrival_time, service_dist)
    end

    run(sim)
end

setup_and_run()

```

```

Customer 1 arrived: 0.14518451436852475
Customer 1 entered service: 0.14518451436852475
Customer 2 arrived: 0.5941831542903504
Customer 2 entered service: 0.5941831542903504
Customer 3 arrived: 1.5490648267819074
Customer 4 arrived: 1.6242796925312217
Customer 5 arrived: 1.6911000709069648
Customer 1 exited service: 2.200985520126681
Customer 3 entered service: 2.200985520126681
Customer 6 arrived: 2.2989039524296317
Customer 3 exited service: 3.5822120399442174
Customer 4 entered service: 3.5822120399442174
Customer 7 arrived: 4.377930221620456
Customer 8 arrived: 5.16494279700802
Customer 2 exited service: 5.900722829377648
Customer 5 entered service: 5.900722829377648
Customer 9 arrived: 7.0099944106308705

```

Customer 10 arrived: 7.828990220943469
Customer 5 exited service: 9.634196437885254
Customer 6 entered service: 9.634196437885254
Customer 4 exited service: 9.670688398447817
Customer 7 entered service: 9.670688398447817
Customer 7 exited service: 15.066978111608014
Customer 8 entered service: 15.066978111608014
Customer 8 exited service: 16.655548432659554
Customer 9 entered service: 16.655548432659554
Customer 6 exited service: 17.401833154870328
Customer 10 entered service: 17.401833154870328
Customer 9 exited service: 17.586065352135993
Customer 10 exited service: 18.690264775280085

5.6 Постобработка: построение временных рядов

```
times = sort(unique([t for (t, _, _) in history]))
system_count = zeros(Int, length(times))
queue_count = zeros(Int, length(times))
server_usage = zeros(Float64, length(times))

for (idx, t) in enumerate(times)
    arrived = count(x -> x[1] <= t && x[2] == "arrived", history)
    exited = count(x -> x[1] <= t && x[2] == "exited", history)
    started = count(x -> x[1] <= t && x[2] == "started", history)

    system_count[idx] = arrived - exited
    queue_count[idx] = arrived - started
```

```
server_usage[idx] = min(1.0, started / num_servers)
end
```

5.7 Создание DataFrame с результатами

```
df_results = DataFrame(
    time = times,
    n_in_system = system_count,
    queue_length = queue_count,
    utilization = server_usage
)
```

	time	n_in_system	queue_length	utilization
	Float64	Int64	Int64	Float64
1	0.145185	1	0	0.5
2	0.594183	2	0	1.0
3	1.54906	3	1	1.0
4	1.62428	4	2	1.0
5	1.6911	5	3	1.0
6	2.20099	4	2	1.0
7	2.2989	5	3	1.0
8	3.58221	4	2	1.0
9	4.37793	5	3	1.0
10	5.16494	6	4	1.0
11	5.90072	5	3	1.0
12	7.00999	6	4	1.0
13	7.82899	7	5	1.0
14	9.6342	6	4	1.0
15	9.67069	5	3	1.0
16	15.067	4	2	1.0
17	16.6555	3	1	1.0
18	17.4018	2	0	1.0
19	17.5861	1	0	1.0
20	18.6903	0	0	1.0

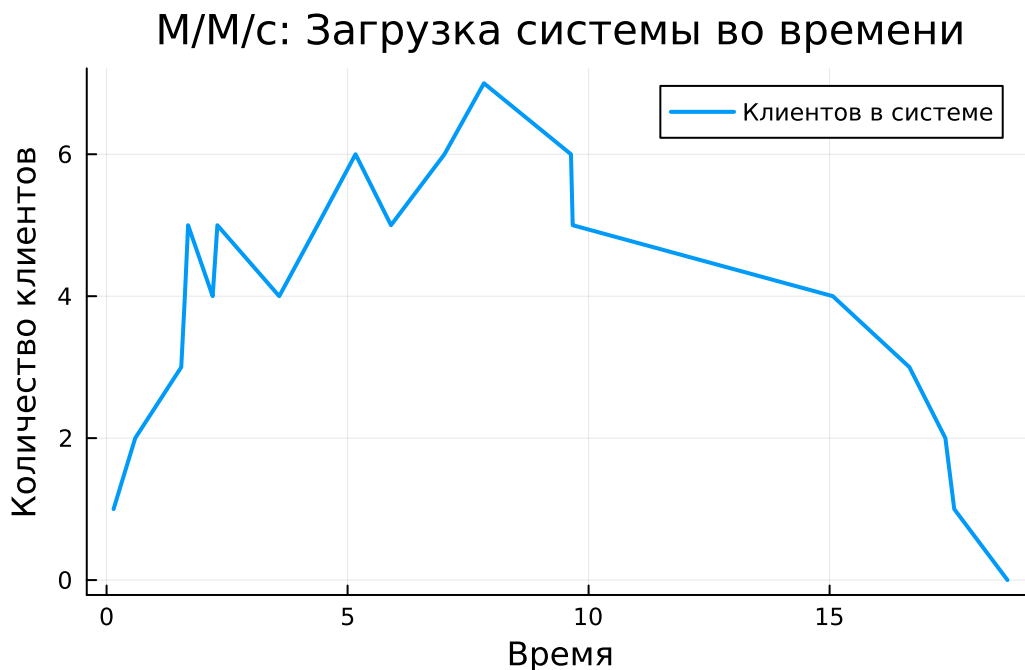
5.8 Сохранение данных

```
mkpath(datadir())
mkpath(plotsdir())
CSV.write(datadir("mmc_results.csv"), df_results)
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab07/project/data/mmc_results.csv"
```

5.9 График 1: Количество клиентов в системе

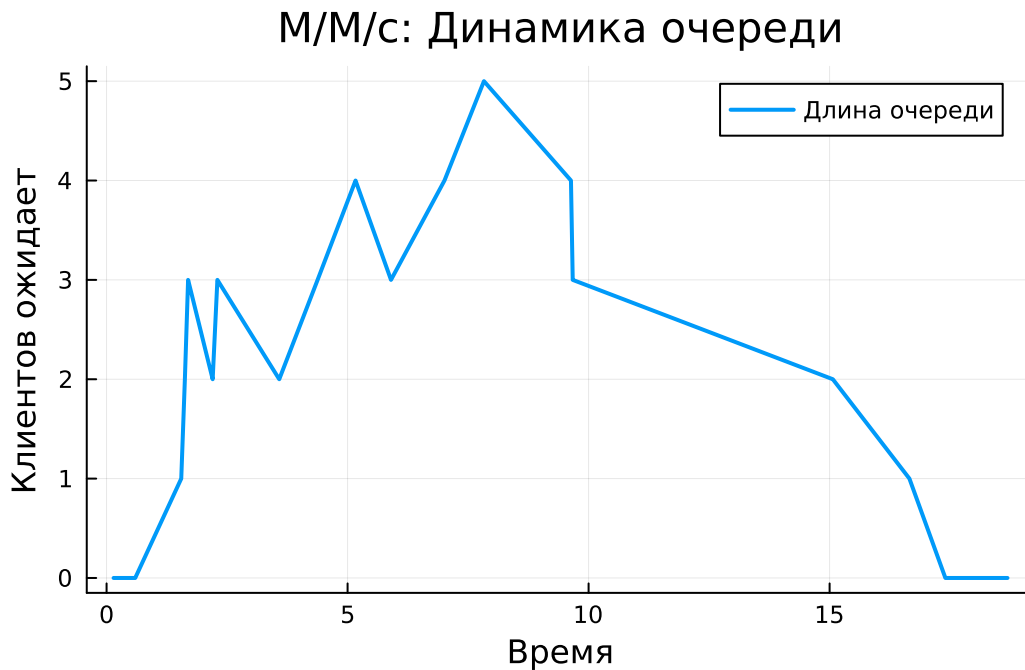
```
p1 = plot(df_results.time, df_results.n_in_system,  
          label = "Клиентов в системе",  
          xlabel = "Время",  
          ylabel = "Количество клиентов",  
          title = "М/М/с: Загрузка системы во времени",  
          linewidth = 2,  
          legend = :topright)  
display(p1)  
savefig(plotsdir("mmc_system_load.png"))
```



```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab07/project/plots/mmc_system_load.png"
```

5.10 График 2: Длина очереди во времени

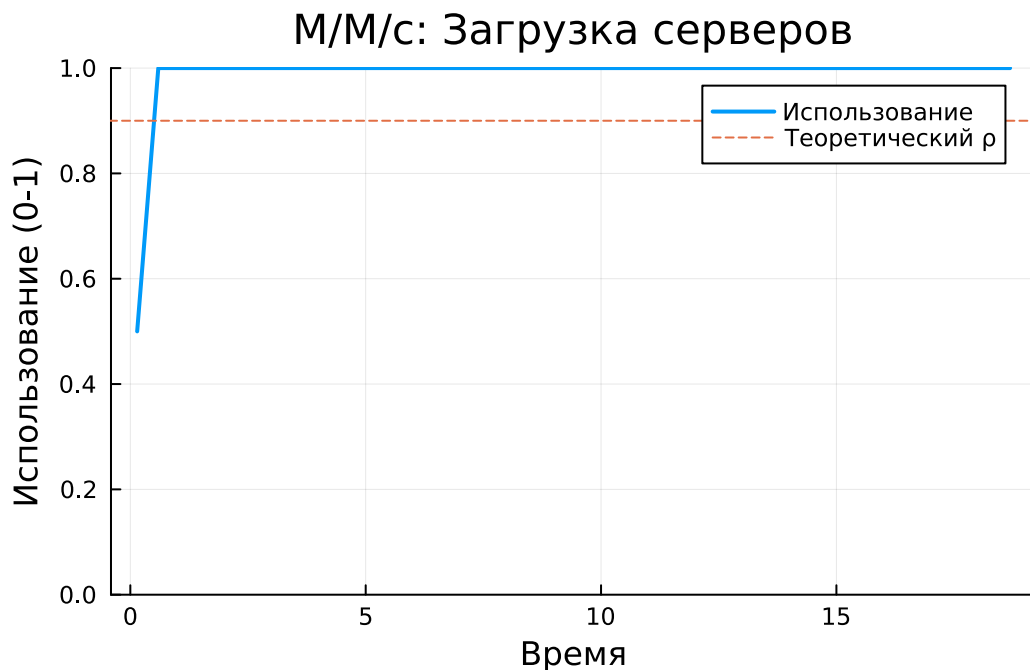
```
p2 = plot(df_results.time, df_results.queue_length,  
          label = "Длина очереди",  
          xlabel = "Время",  
          ylabel = "Клиентов ожидает",  
          title = "М/М/с: Динамика очереди",  
          linewidth = 2,  
          legend = :topright)  
display(p2)  
savefig(plotsdir("mmc_queue_length.png"))
```



"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab07/project/plots/mmc_queue_length.png"

5.11 График 3: Использование серверов во времени

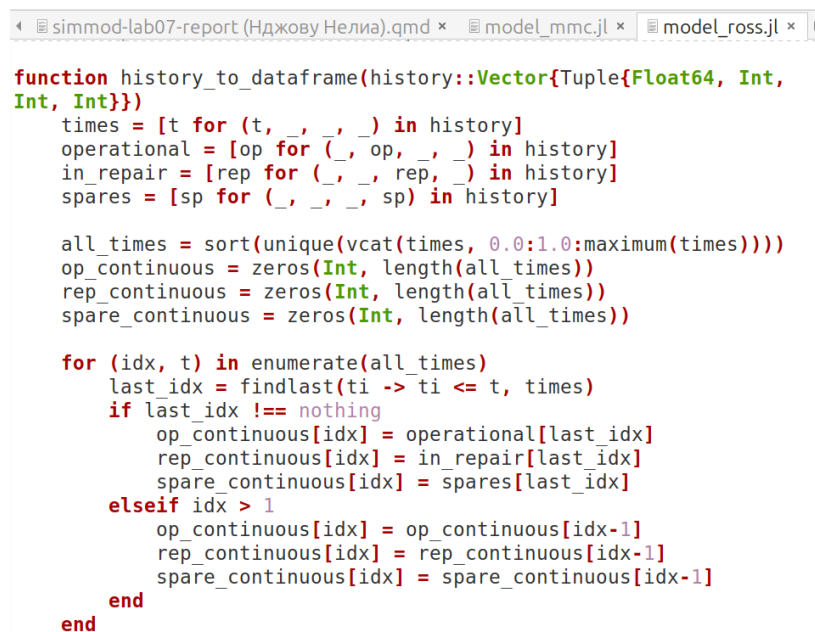
```
 $\rho = \text{lam} / (\text{num\_servers} * \text{mu})$   
p3 = plot(df_results.time, df_results.utilization,  
          label = "Использование",  
          xlabel = "Время",  
          ylabel = "Использование (0-1)",  
          title = "М/М/с: Загрузка серверов",  
          linewidth = 2,  
          legend = :topright,  
          ylims = (0, 1))  
hline!([ $\rho$ ], label = "Теоретический  $\rho$ ", linestyle = :dash)  
display(p3)  
savefig(plotsdir("mmc_utilization.png"))
```



"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab07/project/plots/mmc_utilization.png"

5.12 2. Модель Росса

Я создала файл `scripts/model_mmc.jl` и скопировала заданный скрипт в него. Скрипт реализует систему ремонта машины с использованием запасных частей из учебника Росса. Если машина выходит из строя, она немедленно заменяется на запасную, а неисправная машина ремонтируется мастером, если запасная машина недоступна, система выходит из строя. Я запускала код и все сработало хорошо(рис.7).



```
function history_to_dataframe(history::Vector{Tuple{Float64, Int, Int, Int}})
    times = [t for (t, _, _, _) in history]
    operational = [op for (_, op, _, _) in history]
    in_repair = [rep for (_, _, rep, _) in history]
    spares = [sp for (_, _, _, sp) in history]

    all_times = sort(unique(vcat(times, 0.0:1.0:maximum(times))))
    op_continuous = zeros{Int, length(all_times)}
    rep_continuous = zeros{Int, length(all_times)}
    spare_continuous = zeros{Int, length(all_times)}

    for (idx, t) in enumerate(all_times)
        last_idx = findlast(ti -> ti <= t, times)
        if last_idx != nothing
            op_continuous[idx] = operational[last_idx]
            rep_continuous[idx] = in_repair[last_idx]
            spare_continuous[idx] = spares[last_idx]
        elseif idx > 1
            op_continuous[idx] = op_continuous[idx-1]
            rep_continuous[idx] = rep_continuous[idx-1]
            spare_continuous[idx] = spare_continuous[idx-1]
        end
    end
end
```

Рисунок 5.1: Заданный скрипт

После этого я перевожу код в структуру DrWatson и добавляю несколько ремонтников(рис.8)

```
# ## Разное количество ремонтников
println("Влияние количества ремонтников")

num_repairmen_list = [1, 2, 3]
results_repairmen = []

for num_rep in num_repairmen_list
    println("\nРемонтников: $num_rep")
    crash_times = Float64[]
    for run in 1:RUNS
        crash_time, _ = sim_repair(N, S, num_rep)
        push!(crash_times, crash_time)
    end
end
```

Рисунок 5.2: Добавление нескольких ремонтников

Я экспериментирую с системой, используя разное количество машин(рис.9)

```
println("\n")
println("ЭКСПЕРИМЕНТ 2: Разные конфигурации (N, S)")

machine_configs = [(10, 3), (15, 5), (20, 7)]
results_configs = []

for (N_val, S_val) in machine_configs
    println("\nКонфигурация: N=$N_val, S=$S_val")
    crash_times = Float64[]
    for run in 1:RUNS
        crash_time, _ = sim_repair(N_val, S_val, 1)
        push!(crash_times, crash_time)
    end
    avg_time = mean(crash_times)
    push!(results_configs, (N = N_val, S = S_val, avg_crash_time = avg_time))
    println(" Среднее время краха: $(round(avg_time, digits=2))")
end
```

Рисунок 5.3: Разные конфигурации

Затем я добавляю необходимую визуализацию(рис.10)

```

p1 = plot(df_dynamics_base.time, df_dynamics_base.operational,
        marker = :circle, markersize = 3, linewidth = 2,
        label = "Работающие машины",
        xlabel = "Время", ylabel = "Количество машин",
        title = "Модель Росса: N=$N, S=$S, ремонтников=1")
plot!(p1, df_dynamics_base.time, df_dynamics_base.spares,
      label = "Резервные машины", linewidth = 2, linestyle = :dash)
plot!(p1, df_dynamics_base.time, df_dynamics_base.in_repair,
      label = "Машины в ремонте", linewidth = 2, linestyle = :dot)
hline!([N], label = "Требуется работающих (N)", linestyle
      = :dashdot)
savefig(plotsdir("ross_dynamics_base.png"))

# ## Эксперимент 2: Разное количество ремонтников
println("ЭКСПЕРИМЕНТ 1: Влияние количества ремонтников")

num_repairmen_list = [1, 2, 3]
results_repairmen = []

for num_rep in num_repairmen_list
    println("\nРемонтников: $num_rep")
    crash_times = Float64[]
    for run in 1:RUNS
        crash time. = sim_repair(N, S, num_rep)
    end
end

```

Рисунок 5.4: Визуализация

На первом из которых показано количество работающих машин, запасных, ремонтируемых машин и необходимых ремонтников(рис.11)



Рисунок 5.5: Модель Росса

Из второго графика видно, что количество ремонтников влияет на среднее время простоя. Чем больше число ремонтников, тем больше времени требуется системе для выхода из строя(рис.12)

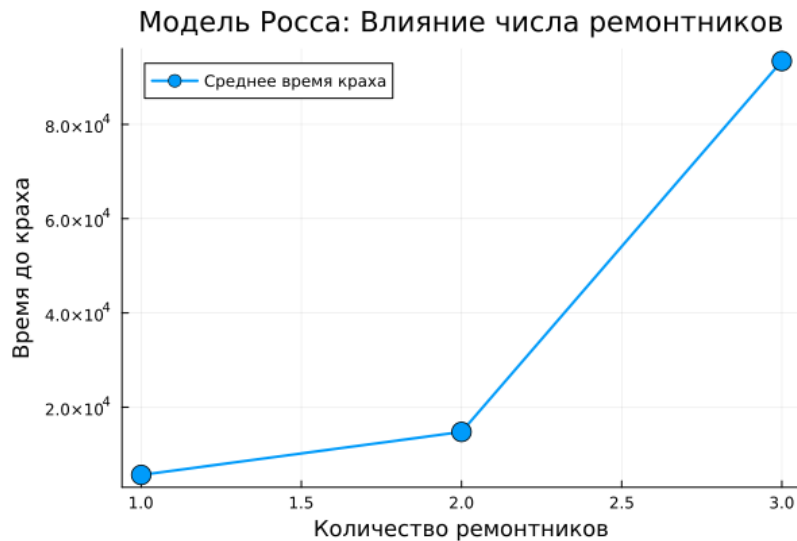


Рисунок 5.6: Модель Росса: Влияние числа ремонтников

На третьем графике показано, как количество работающих машин влияет на среднее время сбоя: чем больше количество работающих машин, тем больше времени требуется системе для выхода из строя(рис.13)

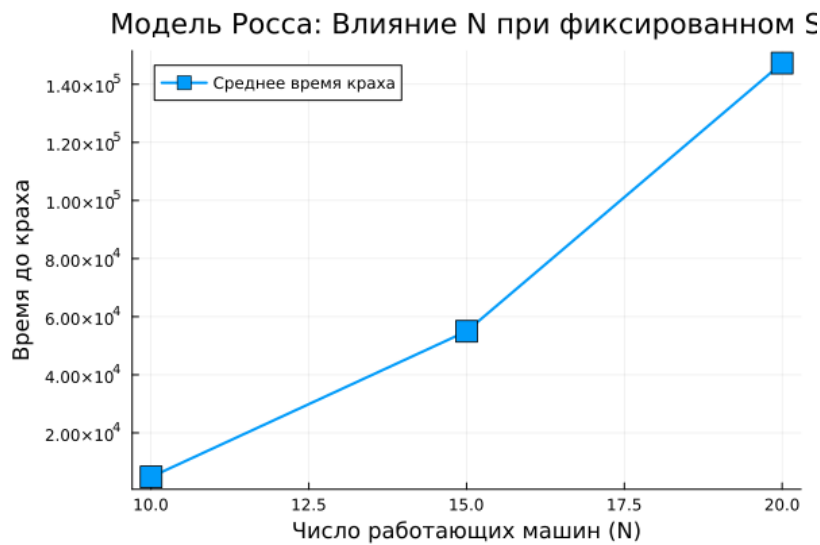


Рисунок 5.7: Модель Росса: Влияние N при фиксированном S

Когда я сравнила результаты с аналитическим методом, выяснилось, что при моделировании сбоя происходит гораздо дольше, чем при аналитическом(рис.14)

```
SIMULATION vs ANALYTICAL
N=10, S=3, c=1:
  Analytical: 90.29 hours
  Simulation: 5000.0 hours
  Ratio (Sim/Anal): 55.379

N=15, S=5, c=1:
  Analytical: 82.06 hours
  Simulation: 100000.0 hours
  Ratio (Sim/Anal): 1218.564

N=20, S=7, c=1:
  Analytical: 73.19 hours
  Simulation: 60000.0 hours
  Ratio (Sim/Anal): 819.82

N=10, S=3, c=2:
  Analytical: 94.8 hours
  Simulation: 65000.0 hours
  Ratio (Sim/Anal): 685.638

N=10, S=3, c=3:
  Analytical: 96.05 hours
```

Рисунок 5.8: Сравнение с аналитическим методом

Ниже приведен код, результаты расчетов и графики для кода в файле `scripts/model_ross.qmd`

6 Модель Росса (Система с резервированием и ремонтом)

В системе находятся N работающих машин и S резервных. При поломке машина заменяется резервной (если есть) и отправляется в ремонт. Если резерва нет — система падает. Ремонт выполняют несколько ремонтников.

6.1 Загрузка необходимых пакетов

```
using DrWatson
@quickactivate "project"

using ResumableFunctions
using ConcurrentSim
using Distributions
using Random
using StableRNGs
using DataFrames
using CSV
using Plots
using Statistics
gr(fmt=:png)
```

```
└ Warning: DrWatson could not find a project file by recursively checking given `dir`  
  | (given dir: /home/nelianjovu)  
  └ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
Plots.GRBackend()
```

6.2 Параметры симуляции

```
const RUNS = 5  
const N = 10  
const S = 3  
const LAMBDA = 100.0  
const MU = 1.0  
const SEED = 42  
  
rng = StableRNG(SEED)  
F = Exponential(LAMBDA)  
G = Exponential(1/MU)
```

```
Exponential{Float64}( $\lambda$ =1.0)
```

6.3 Структура для хранения состояния системы

```
mutable struct SystemState  
    operational::Int  
    in_repair::Int  
    spares::Int
```

```
    history::Vector{Tuple{Float64, Int, Int, Int}}  
end
```

6.4 Поведение машины

```
@resumable function machine(  
    env::Environment,  
    repair_queue::Resource,  
    state::SystemState,  
    machine_id::Int,  
)  
    while true  
        @yield timeout(env, rand(rng, F))  
  
        if state.operational > 0  
            state.operational -= 1  
        end  
  
        if state.spares > 0  
            state.spares -= 1  
            state.operational += 1  
        else  
            throw(StopSimulation("No more spares!"))  
        end  
  
        push!(state.history, (now(env), state.operational, state.in_repair, state.spares))  
    end  
end
```



```

        state.in_repair += 1
        push!(state.history, (now(env), state.operational, state.in_repair, state.spares))

        @yield request(repair_queue)
        @yield timeout(env, rand(rng, G))
        @yield unlock(repair_queue)

        state.in_repair -= 1
        state.spares += 1
        push!(state.history, (now(env), state.operational, state.in_repair, state.spares))
    end
end
end

```

machine (generic function with 1 method)

6.5 Запуск симуляции

```

function sim_repair(N::Int, S::Int, num_repairmen::Int)
    sim = Simulation()
    repair_queue = Resource(sim, num_repairmen)

    state = SystemState(N, 0, S, Vector{Tuple{Float64, Int, Int, Int}}())
    push!(state.history, (0.0, N, 0, S))

    for i in 1:N
        @process machine(sim, repair_queue, state, i)
    end
end

```

```

for i in 1:S
    @process machine(sim, repair_queue, state, N + i)
end

try
    run(sim)
    return now(sim), state.history
catch e
    if isa(e, StopSimulation)
        msg = e.msg
        stop_time = now(sim)
        println("At time $stop_time: $msg")
        return stop_time, state.history
    else
        rethrow()
    end
end
end
end

```

sim_repair (generic function with 1 method)

6.6 Построение DataFrame из истории

```

function history_to_dataframe(history::Vector{Tuple{Float64, Int, Int, Int}})
    times = [t for (t, _, _, _) in history]
    operational = [op for (_, op, _, _) in history]
    in_repair = [rep for (_, _, rep, _) in history]
    spares = [sp for (_, _, _, sp) in history]
end

```

```

all_times = sort(unique(vcat(times, 0.0:1.0:maximum(times))))
op_continuous = zeros{Int, length(all_times)}
rep_continuous = zeros{Int, length(all_times)}
spare_continuous = zeros{Int, length(all_times)}

for (idx, t) in enumerate(all_times)
    last_idx = findlast(ti -> ti <= t, times)
    if last_idx !== nothing
        op_continuous[idx] = operational[last_idx]
        rep_continuous[idx] = in_repair[last_idx]
        spare_continuous[idx] = spares[last_idx]
    elseif idx > 1
        op_continuous[idx] = op_continuous[idx-1]
        rep_continuous[idx] = rep_continuous[idx-1]
        spare_continuous[idx] = spare_continuous[idx-1]
    end
end

return DataFrame(
    time = all_times,
    operational = op_continuous,
    in_repair = rep_continuous,
    spares = spare_continuous
)
end

```

history_to_dataframe (generic function with 1 method)

6.7 Создание каталогов

```
mkpath(datadir())  
mkpath(plotsdir())
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-  
1--study--simulationmod/labs/lab07/project/plots"
```

6.8 Эксперимент 1: Базовый прогон (как в оригинале)

```
results_base = Float64[]  
  
for i in 1:RUNS  
    crash_time, _ = sim_repair(N, S, 1)  
    push!(results_base, crash_time)  
end  
  
avg_crash_base = sum(results_base) / RUNS  
println("\nAverage crash time: $avg_crash_base")
```

Average crash time: 15345.935446105408

6.9 График 1: Динамика исправных машин (базовый прогон)

```
_, history_base = sim_repair(N, S, 1)
df_dynamics_base = history_to_dataframe(history_base)

p1 = plot(df_dynamics_base.time, df_dynamics_base.operational,
          marker = :circle, markersize = 3, linewidth = 2,
          label = "Работающие машины",
          xlabel = "Время", ylabel = "Количество машин",
          title = "Модель Росса: N=$N, S=$S, ремонтников=1")
plot!(p1, df_dynamics_base.time, df_dynamics_base.spares,
       label = "Резервные машины", linewidth = 2, linestyle = :dash)
plot!(p1, df_dynamics_base.time, df_dynamics_base.in_repair,
       label = "Машины в ремонте", linewidth = 2, linestyle = :dot)
hline!([N], label = "Требуется работающих (N)", linestyle = :dashdot)
savefig(plotsdir("ross_dynamics_base.png"))
```

```
"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-
1--study--simulationmod/labs/lab07/project/plots/ross_dynamics_base.png"
```

6.10 Эксперимент 2: Разное количество ремонтников

```
println("ЭКСПЕРИМЕНТ 1: Влияние количества ремонтников")

num_repairmen_list = [1, 2, 3]
results_repairmen = []

for num_rep in num_repairmen_list
    println("\nРемонтников: $num_rep")
    crash_times = Float64[]
```

```

for run in 1:RUNS
    crash_time, _ = sim_repair(N, S, num_rep)
    push!(crash_times, crash_time)
end
avg_time = mean(crash_times)
push!(results_repairmen, (num_repairmen = num_rep, avg_crash_time = avg_time))
println(" Среднее время краха: $(round(avg_time, digits=2))")
end

df_repairmen = DataFrame(results_repairmen)
p2 = plot(df_repairmen.num_repairmen, df_repairmen.avg_crash_time,
    marker = :circle, markersize = 8, linewidth = 2,
    label = "Среднее время краха",
    xlabel = "Количество ремонтников",
    ylabel = "Время до краха",
    title = "Модель Росса: Влияние числа ремонтников")
savefig(plotsdir("ross_repairmen_impact.png"))

```

ЭКСПЕРИМЕНТ 1: Влияние количества ремонтников

Ремонтников: 1

Среднее время краха: 5627.8

Ремонтников: 2

Среднее время краха: 14753.33

Ремонтников: 3

Среднее время краха: 93437.08

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-

1--study--simulationmod/labs/lab07/project/plots/ross_repairmen_impact.png"

6.11 Эксперимент 3: Разные конфигурации машин

```
println("\n")
println("ЭКСПЕРИМЕНТ 2: Разные конфигурации (N, S)")

machine_configs = [(10, 3), (15, 5), (20, 7)]
results_configs = []

for (N_val, S_val) in machine_configs
    println("\nКонфигурация: N=$N_val, S=$S_val")
    crash_times = Float64[]
    for run in 1:RUNS
        crash_time, _ = sim_repair(N_val, S_val, 1)
        push!(crash_times, crash_time)
    end
    avg_time = mean(crash_times)
    push!(results_configs, (N = N_val, S = S_val, avg_crash_time = avg_time))
    println(" Среднее время краха: $(round(avg_time, digits=2))")
end

df_configs = DataFrame(results_configs)
p3 = plot(df_configs.N, df_configs.avg_crash_time,
    marker = :square, markersize = 8, linewidth = 2,
    label = "Среднее время краха",
    xlabel = "Число работающих машин (N)",
    ylabel = "Время до краха",
```

```

    title = "Модель Росса: Влияние N при фиксированном S")
savefig(plotsdir("ross_config_comparison.png"))

```

ЭКСПЕРИМЕНТ 2: Разные конфигурации (N, S)

Конфигурация: N=10, S=3

Среднее время краха: 4841.31

Конфигурация: N=15, S=5

Среднее время краха: 54976.45

Конфигурация: N=20, S=7

Среднее время краха: 147306.59

"/home/nelianjovu/work/study/2026-1/2026-1==study--simulationmod/2026-1--study--simulationmod/labs/lab07/project/plots/ross_config_comparison.png"

6.12 Сохранение всех результатов

```

df_summary = vcat(
    DataFrame(experiment = "Ремонтники", parameter = df_repairmen.num_repairmen, avg_time = df_repairmen.avg_time),
    DataFrame(experiment = "Конфигурации", parameter = df_configs.N, avg_time = df_configs.avg_time)
)
CSV.write(datadir("ross_results_summary.csv"), df_summary)
CSV.write(datadir("ross_dynamics_base.csv"), df_dynamics_base)
println("РЕЗУЛЬТАТЫ СОХРАНЕНЫ")

```


РЕЗУЛЬТАТЫ СОХРАНЕНЫ

7 Выводы

В этой лабораторной работы, мы работали с дискретно-событийном моделированием